

FOC Sine ESC CAN Communication Protocol V3.0

Contents

1	Preface.....	3
2	Protocol definition	4
2.1	CAN communication protocol.....	4
2.2	Frame Format.....	4
2.2.1	ID field	4
2.2.2	Data field	5
3	ESC communication protocol (in common use)	8
3.1.1	Broadcast throttle 1	8
3.1.2	Broadcast throttle 2	8
3.1.3	Set the status upload period	9
3.1.4	ESC status information 1	9
3.1.5	ESC status information 2	10
3.1.6	ESC status information 3	10
4	ESC communication protocol (PX4 or APM flight control)	11
4.1.1	NodeStatus instruction	11
4.1.2	RawCommand instruction	11
4.1.3	Status instruction	13
5	ESC communication protocol (else)	14
5.1.1	Obtain ESC hardware information	14
5.1.2	Set the digital throttle	15
5.1.3	Get real-time data.....	16
5.1.4	Set speed	16
5.1.5	Set the digital throttle 2	18
5.1.6	Motor open loop rotation	19
5.1.7	Motor Beep	19
5.1.8	ESC LED Control	20
6	ESC communication protocol (Encoder support required)	21
6.1.1	The mechanical Angle setting was enabled	21
6.1.2	Set the rotating mechanical Angle.....	21
6.1.3	Set the active upload mechanical Angle	22
6.1.4	ESC broadcast mechanical Angle	22

6.1.5	Calibrate the motor position sensor	22
6.1.6	Obtain the status of the calibrated motor position sensor	23
7	ESC addressing	23
7.1.1	ESC node parameters	23
7.1.2	ESC communication addressing	24
8	ESC Firmware upgrade	25
8.1	Firmware file composition	25
8.2	Device restart	26
8.3	Keep the Bootloader status	26
8.4	Start upgrade	26
8.5	Firmware transfer	27
8.6	End Upgrade	27
8.7	Firmware upgrade flowchart	28
9	Appendix 1 Definition of Error codes	29
9.1	Self-check error code	29
9.2	Run code error	30

1 Preface

The CAN communication format is compatible with the DroneCAN(UAVCAN V0.9) protocol, and multi-byte data is arranged with the low position in front and the high position in back. Baud rate: 1000kbps.

More details about the DroneCAN protocol can be found at:

<https://dronecan.github.io/>

2 Protocol definition

2.1 CAN communication protocol

The frame format of CAN is extended frame, and the frame type is data frame. Multiple bytes of data are arranged with the low position in front and the high position in back. CAN Data Length The DLC should be the total length of valid data.

2.2 Frame Format

2.2.1 ID field

The extended frame ID only uses 28 to 0 bits, and the default node ID is 0x20. If the node ID is set by the user, the node ID is the newly set node ID.

2.2.1.1 Broadcast message frame ID

Definition				Priority					Data ID																Frame type								
																									Source node ID								
Bit area	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Range value	0			0~32					20000~21000																0	1~127							
Byte	3								2								1								0								

2.2.1.2 Anonymous broadcast message frame ID

Definition				Priority					Data ID														Frame type									
									Identity ID															Source node ID								
Bit area	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Range value	0			0~32					使用 CRC 的中间14位														0~3		0	0						
Byte	3								2						1						0											

2.2.1.3 Service frame ID

Definition				Priority					Data ID								Service type								Frame type									
																	Target Node ID								Source node ID									
Bit area	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
Range value	0			0~32					0~225									1~127								1	1~127							
Byte	4					3								2								1												

2.2.2 Data field

The frame data field of CAN consists of the actual data and the trailing bytes

Data field Name	Actual Data0~N	Tail Byte
Description	N The value ranges from 0 to 7	Details check Tail Byte Definition

Trailing byte definition:

Definition	Stop Output	Stop Output	Switching output	Transmission ID				
Bit area	7	6	5	4	3	2	1	0
Range value	0~1	0~1	1	0~31				

Transmission ID generally means that the same instruction data is counted in the transmission process, and the field can be ignored when implementing the protocol.

2.2.2.1 Single frame transmission

If the data transmitted does not exceed 7 bytes, you can directly use the single-frame transmission method.

It is recommended to use single frame transmission for transmission.

The transmission data structure is as follows:

Data 0~ N	Tail Byte
N The value ranges from 0 to 7	Start transmitting, always 1 Stop transmission, always 1 Switch identifier, always 0

2.2.2.2 Multiframe transmission

If more than 7 bytes of data are transferred, multi-frame transmission is required, in which the data CRC must be included in the first frame of data.

Trailing byte definition:

	Data0	Data1	Data2~6	Tail Byte
The first frame	The data CRC is 8 bits lower	The data CRC is 8 bits higher	5 bytes	Start transmitting, always 1 Stop transmission, always 1 Switch identifier, always 0

	Data0~7	Tail Byte
The Nth Frame	7 bytes	Start transmitting, always 0 Stop transmission, always 1 Toggle identification, with the first frame being 0 and then alternating between 0 and 1

	Data0~N	Tail Byte
The last frame	N The value ranges from 0 to 7	Start transmitting, always 0 Stop transmission, always 1 Toggle identification, with the first frame being 0 and then alternating between 0 and 1

2.2.2.3 CRC calculation of multi-frame transmission data

The data CRC calculation is calculated based on the valid data transmitted, and the calculated signature has a type signature of the data ID (uint64, small-end sort).

Data signature (uint64), small end	N bytes of transmitted data
------------------------------------	-----------------------------

Calculation reference code:

```
uint16_t calcCRC(uint64_t data_signature, uint8_t* payload, uint16_t payload_len)
{
    uint16_t crc = 0xFFFFU;

    //add signature
    for (uint16_t shift_val = 0; shift_val < 64; shift_val = (uint16_t)(shift_val + 8U))
    {
        crc = crc16AddByte(crc, (uint8_t) (data_signature >> shift_val));
    }

    //add data_payload
    for(uint16_t i = 0; i < payload_len; i++)
    {
        crc = crc16AddByte(crc, payload[i], payload_len);
    }
    return crc;
}

uint16_t crc16AddByte(uint16_t crc_val, uint8_t byte)
{
    crc_val ^= (uint16_t) ((uint16_t) (byte) << 8U);
    for (uint8_t j = 0; j < 8; j++)
    {
        if (crc_val & 0x8000U)
        {
            crc_val = (uint16_t) ((uint16_t) (crc_val << 1U) ^ 0x1021U);
        }
        else
        {
            crc_val = (uint16_t) (crc_val << 1U);
        }
    }
    return crc_val;
}
```


3 ESC communication protocol (in common use)

3.1.1 Broadcast throttle 1

	Broadcast message
Data ID	20010
Data signature	0x5F5282808C0EAB76
Data format	uint8 esc_id uint16 pwm_us //unit 0.1us
Example	The priority is 0, the host ID is 1, and the throttle of power tuner 0x20 is set to 1300us CAN ID CAN DATA 004E2A01 0x20 0xC8 0x32 0xC0

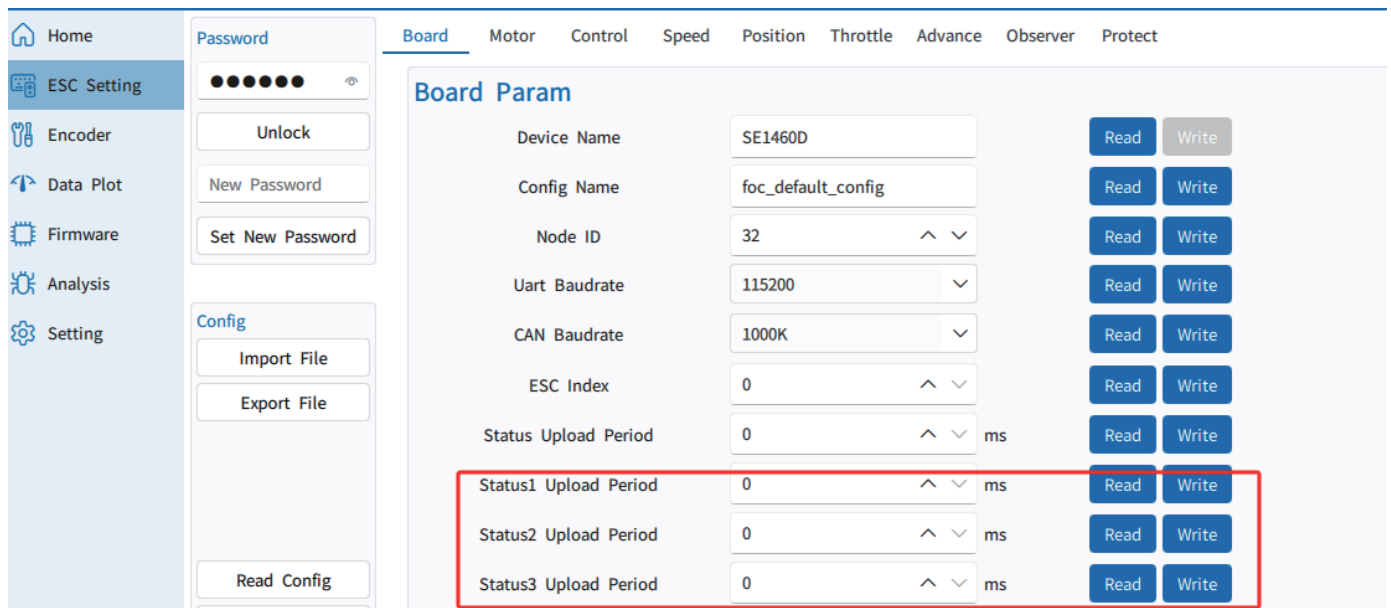
3.1.2 Broadcast throttle 2

	Broadcast message
Data ID	20011
Data signature	0xAA4B857FBD34B46F
Data format	uint8 esc_id_1 uint16 pwm_us_2 //unit 0.1us uint8 esc_id_2 uint16 pwm_us_2 //unit 0.1us
Example	The priority is 0, the host ID is 1, and the throttle of the power-off 0x20 is set to 1300us. The throttle of the power-off 0x21 is set to 1300us CAN ID CAN DATA 004E2B01 0x20 0xC8 0x32 0x21 0xC8 0x32 0xC0

3.1.3 Set the status upload period

	Request	Response
Data ID	232	
Data signature	0x997BF9DE5FB3D4D7	
Data format	uint8 opcode #0,read, 1,write;2,Write and save to flash uint16 status_msg_id #The supported broadcast messages are 20022,20023,20024 uint16 upload_period_ms #unit ms, Value range:0~10000	uint8 opcode_result # 0: indicates success. 1: indicates the ID of the unsupported message. 2, indicates that the motor is running and cannot be saved to flash. uint16 status_msg_id uint16 upload_period_ms
Example	Set priority 0, host ID 1, Power-off ID 0x20, and power-off status information 1 Upload period to 1000ms CAN ID CAN DATA 00E8A081 0x02 0x36 0x4E 0xE8 0x03 0xC0 00E801A0 0x00 0x36 0x4E 0xE8 0x03 0xC0	

The status upload cycle can be performed directly from the host computer.



3.1.4 ESC status information 1

	Broadcast message
Data ID	20022

Data signature	0x9309D0AE1328B7AE
Data format	Int24 speed # rpm Motor speed uint16 recv_pwm # 0.1us ESC receiving PWM uint16 comm_pwm #0.1us ESC receives the communication PWM
Example	Priority 0, ESC ID 为 0x20 CAN ID CAN DATA 184E3620 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0xC0

3.1.5 ESC status information 2

	Broadcast message
Data ID	20023
Data signature	0xE43AE5E5278F7D32
Data format	uint16 voltage # 0.1v Bus voltage int16 bus_current #0.1A Bus current int16 current # 0.1A Motor line current uint8 v_modulation #Modulation ratio
Frame ID field	Priority 0, ESC ID 为 0x20 CAN ID CAN DATA 184E3720 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0xC0

3.1.6 ESC status information 3

	Broadcast message
Data ID	20024
Data signature	0xD390FA7B6D685BF6
Data format	uint8 esc_index #ESC number uint8 mos_temp # degrees Celsius,MOS temperature: The temperature bias is -50, that is, temperature= (int16)mos_temp - 50 uint8 cap_temp # degrees Celsius,Capacitor temperature: The temperature bias is -50, that is, temperature= (int16)mos_temp - 50 uint8 mcu_temp # degrees Celsius,MCU temperature, temperature offset is -50,

	that is, temperature= (int16)mos_temp - 50 uint8 motor_temp # degrees Celsius, Motor temperature, temperature bias is 50, that is, temperature= (int16)mos_temp - 50 uint16 running_error
Example	Priority 0, ESC ID 为 0x20 CAN ID CAN DATA 184E3820 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0xC0

4 ESC communication protocol (PX4 or APM flight control)

Currently ESC can support the ESC command protocol for PX4 or APM flight control, but special firmware is required to support this feature.

4.1.1 NodeStatus instruction

	Broadcast message
Data ID	341
Data signature	0x217F5C87D7EC951D
Data format	uint32 uptime_sec uint2 health uint3 mode uint3 sub_mode uint16 vendor_specific_status_code #ESC self-checks the error code
Example	CAN ID CAN DATA

NodeStatus instruction is broadcast once at a fixed 1000ms.

4.1.2 RawCommand instruction

	Broadcast message
--	-------------------

The RawCommand command must be used with the power commission number configured. For example, cmd[0] is the value of the command whose number is 0, and cmd[1] is the value of the command whose number is 1. The switchboard number can be configured in the upper computer.

The PWM of the 0-8191 map is "(normal PWM start value + hysteresis throttle value)" ~ normal PWM end value].

The value of the 0-8191 map is (1100+10) to 1900

Board Motor Control Speed Position Throttle Advance Observer Protect

Throttle Param

Pwm Period Tolerance	100	^ v	us	Read	Write
Pwm Lost Time	1000	^ v	ms	Read	Write
Hyst Pwm	10	^ v	us	Read	Write
Throttle Type	Normal	v		Read	Write
Pwm Curve Type	Linear Speed	v		Read	Write
Normal Pwm Start	1100	^ v	us	Read	Write
Normal Pwm End	1900	^ v	us	Read	Write
Positive Pwm Start	1525	^ v	us	Read	Write
Positive Pwm End	1900	^ v	us	Read	Write

4.1.3 Status instruction

	Broadcast message	
Data ID	1034	
Data signature	0xE43AE5E5278F7D32	
Data format	uint32 error_count	# The fault code is parsed according to the error code definition in Appendix 1
	float16 voltage	# Volt
	float16 current	# Ampere. Can be negative in case of a regenerative braking.
	float16 temperature	# Kelvin
	int18 rpm	# Negative value indicates reverse rotation
	uint7 power_rating_pct	# Instant demand factor in percent (percent of maximum power); range 0% to 127%.
	uint5 esc_index	
Example	Priority 0, ESC ID 为 0x20 CAN ID CAN DATA	

You can configure the upload period of the status on the upper ESC.

5.1.2 Set the digital throttle

	Request	Response
Data ID	210	
Data signature	0x1653C41DA4E45824	
Data format	uint16 pwm_us //unit 0.1us	uint8 valid_flag
示例	Priority 0, host ID is 1, ESC ID is 0x20, and throttle is set to 1300us CAN ID CAN DATA 00D2A081 0xC8 0x32 0xC0 00D201A0 0x00 0xC0	

5.1.3 Get real-time data

	Request	Response
Data ID	211	
Data signature	0x9AB6F8CCCBF6F016	
Data format	none	uint16 recv_pwm # 0.1us ESC receiving PWM uint16 comm_pwm # 0.1us ESC receives the communication PWM int32 speed # rpm Motor speed int16 current # 0.1A Motor line current int16 bus_current # 0.1A Bus current uint16 voltage # 0.1v Bus voltage uint16 v_modulation # 0.1 Modulation ratio int16 mos_temp # 0.1 degrees Celsius ,Mos temperature int16 cap_temp # 0.1 degrees Celsius ,Capacitance temperature int16 mcu_temp # 0.1 degree Celsius , MCU temperature uint16 running_error #Run error code uint16 selfcheck_error #Self-check error code int16 motor_temp # 0.1 degree Celsius , Motor temperature uint16 time_10ms #10ms ESC running time uint8 resv[3] #Reserved word
Example	Priority 0, host ID is 1, ESC ID is 0x20, CAN ID CAN DATA 00D3A081 0xC0 00D301A0 CRC_L CRC_H Byte1 Byte2 Byte3 Byte4 Byte5 0x80 00D301A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x20 00D301A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x00 00D301A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x20 00D301A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x40	

5.1.4 Set speed

	Request	Response
Data ID	212	
Data signature	0x4461BB2B55B3F410	
Data format	int32 speed #rpm Motor speed	uint16 recv_pwm # 0.1us ESC receiving PWM uint16 comm_pwm #0.1us ESC receives the communication PWM int32 speed # rpm Motor speed int16 current # 0.1A Motor line current int16 bus_current #0.1A Bus current uint16 voltage # 0.1v Bus voltage uint16 v_modulation #0.1 Modulation ratio int16 mos_temp #0.1 degrees Celsius ,Mos temperature int16 cap_temp #0.1 degrees Celsius ,Capacitance temperature int16 mcu_temp #0.1 degree Celsius , MCU temperature uint16 running_error #Run error code uint16 selfcheck_error #Self-check error code int16 motor_temp #0.1 degree Celsius , Motor temperature uint16 time_10ms #10ms ESC running time uint8 resv[3] #Reserved word
Example	The priority is 0, the host ID is 1, the ESC ID is 0x20, and the speed is 504 CAN ID CAN DATA 00D4A081 0xF8 0x01 0x00 0x00 0xC0 00D401A0 CRC_L CRC_H Byte1 Byte2 Byte3 Byte4 Byte5 0x80 00D401A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x20 00D401A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x00 00D401A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x20 00D401A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x40	

5.1.5 Set the digital throttle 2

	Request	Response
Data ID	213	
Data signature	0x81B48C174CDA4C7A	
Data format	uint16 pwm_us //unit 0.1us	uint16 recv_pwm # 0.1us ESC receiving PWM uint16 comm_pwm #0.1us ESC receives the communication PWM int32 speed # rpm Motor speed int16 current # 0.1A Motor line current int16 bus_current #0.1A Bus current uint16 voltage # 0.1v Bus voltage uint16 v_modulation #0.1 Modulation ratio int16 mos_temp #0.1 degrees Celsius ,Mos temperature int16 cap_temp #0.1 degrees Celsius ,Capacitance temperature int16 mcu_temp #0.1 degree Celsius , MCU temperature uint16 running_error #Run error code uint16 selfcheck_error #Self-check error code int16 motor_temp #0.1 degree Celsius , Motor temperature uint16 time_10ms #10ms ESC running time uint8 resv[3] #Reserved word
Example	Priority 0, host ID is 1, ESC ID is 0x20, and throttle is set to 1300us CAN ID CAN DATA 00D5A081 0xC8 0x32 0xC0 00D501A0 CRC_L CRC_H Byte1 Byte2 Byte3 Byte4 Byte5 0x80 00D501A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x20 00D501A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x00 00D501A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x20 00D501A0 Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 Byte7 0x40	

5.1.6 Motor open loop rotation

	Request	Response
Data ID	217	
Data signature	0xE035AD9BC3A39246	
Data format	# 0: Query the rotation status of the motor # 1: Setting Angle controls the motor rotation, setting Angle positive or negative indicates the motor rotation direction, setting speed and current must be positive # 2: Set the speed control motor to continue to rotate, set the speed positive or negative indicates the direction of motor rotation, the current should be positive Angle is invalid #3: Exit the motor open loop rotation function uint8 cmd_id int16 speed # Unit RPM int16 current # Unit 0.1A int16 angle # Unit degree	# 0 End of motor rotation # 1 The motor is spinning # 2 If the conditions are not met, the open-loop rotation function is not performed. # (Description: The electric regulation is normal, the motor is stopped, there is PWM and below the idle throttle, allowing open loop rotation. Open loop mode #automatically exit open loop mode) uint8 state
Example	Priority 0, host ID 1, ESC ID 0x20, set the motor at 10RPM, current 3A, 180° forward from the current position CAN ID CAN DATA 00D9A081 0x01 0x0A 0x00 0x1E 0x00 0XB4 0x00 0xC0 00D901A0 0x01 0xC0	

5.1.7 Motor Beep

	Request	Response
Data ID	214	
Data signature	0x40C65964649CAE65	
Data format	uint8 beep_num # 1-10, Number of sounds #Sound data, the lower 8 bits, is the tone, the value range 0 to 9 #The middle 8 digits are duration, measured in 10ms #The high 8 bits are volume,0 to 100 uint24 beep_data[<=10] #Sound data	uint8 flag #0 success

Example	Priority 0, host ID is 1, ESC ID is 0x20 CAN ID CAN DATA
---------	--

5.1.8 ESC LED Control

	Request	Response
Data ID	214	
Data signature	0x40C65964649CAE65	
Data format	uint8 led_num # Led sequence quantity #LED data #low 8 bits,0: off. 1: red; 2: green; 3, yellow,4: red breath light, 5: green breath light,6, yellow breath light #high 8 bits is time,0~200, unit 50ms uint16 led_data[<=10]	uint8 flag #0 success
Example	Priority 0, host ID is 1, ESC ID is 0x20 CAN ID CAN DATA	

6 ESC communication protocol (Encoder support required)

ESC provides position control for motors that support our encoders.

6.1.1 The mechanical Angle setting was enabled

	Request	Response
Data ID	215	
Data signature	0x8776979B269EFCFF	
Data format	uint8 enable_mech_nagle #Enable the mechanical Angle	uint8 flag #0 Show success
Example	Priority 0, host ID is 1, ESC ID is 0x20 CAN ID CAN DATA 00D7A081 0x01 0xC0 00D701A0 0x00 0xC0	

6.1.2 Set the rotating mechanical Angle

	Request	Response
Data ID	216	
Data signature	0x77ED7522030ED464	
Data format	int16 mech_angle #unit°	uint8 flag #0 Show success
Example	Priority 0, host ID is 1, ESC ID is 0x20, motor to 80° position CAN ID CAN DATA 00D8A081 0x50 0x00 0xC0 00D801A0 0x00 0xC0	

6.1.3 Set the active upload mechanical Angle

	Request	Response
Data ID	231	
Data signature	0x602064459E9039DB	
Data format	#unit hz , 0~400, 0Indicates close upload uint16 upload_hz	# 0 indicates that the data is successful, # 1 indicates that the data is invalid uint8 flag
Example	The priority is 0, the host ID is 1, the ESC ID is 0x20, and the upload frequency is 300hz CAN ID CAN DATA 00E7A081 0x2C 0x01 0xC0 00E701A0 0x00 0xC0	

6.1.4 ESC broadcast mechanical Angle

	Broadcast message										
Data ID	20021										
Data signature	0xA922986F6F2D9C0E										
Data format	int16 mech_angle #unit°										
Example	The priority is 0, the ESC ID is 0x20, and the change of the tail byte is a cyclic increment of the transmission ID. The value ranges from 0 to 31 <table> <tr> <td>CAN ID</td><td>CAN DATA</td></tr> <tr> <td>104E3520</td><td>0x64 0x00 0xC0</td></tr> <tr> <td>104E3520</td><td>0x64 0x00 0xC1</td></tr> <tr> <td>...</td><td></td></tr> <tr> <td>104E3520</td><td>0x64 0x00 0xDF</td></tr> </table>	CAN ID	CAN DATA	104E3520	0x64 0x00 0xC0	104E3520	0x64 0x00 0xC1	...		104E3520	0x64 0x00 0xDF
CAN ID	CAN DATA										
104E3520	0x64 0x00 0xC0										
104E3520	0x64 0x00 0xC1										
...											
104E3520	0x64 0x00 0xDF										

6.1.5 Calibrate the motor position sensor

	Request	Response
Data ID	225	
Data signature	0xB970F68E635BAD53	
Data format	uint16 cmd_id #0x20 Calibrating sensor uint8_t data[len] #len <= 48	#0 indicates that the setting is successful. # Other status is abnormal

		uint8 flag
Example	With priority 0, host ID 1, ESC ID 0x20, and data[0] 1, calibrate is enabled CAN ID CAN DATA 00E1A081 0x20 0x00 0x01 0xC0 00E101A0 0x20 0x00 0xC0	

6.1.6 Obtain the status of the calibrated motor position sensor

	Request	Response
Data ID	226	
Data signature	0xA895FDA3617029D2	
Data format	uint16 cmd_id #0x21 Acquisition state int32_t resv_data #0x00	uint16 cmd_id #0x21 uint8 flag #0 Indicates that data is obtained successfully uint8_t data[len] #len <= 48
Example	Priority 0, host ID is 1, ESC ID is 0x20, data[0] :1. During calibration, 2 is successfully calibrated, 3, calibration fails CAN ID CAN DATA 00E2A081 0x21 0x00 0x00 0x00 0x00 0x00 0x00 0xC0 00E201A0 0x21 0x00 0x00 0x02 0xC0	

7 ESC addressing

The default ID of an ESC is 0x20. ESC address ID is divided into two forms, one is the node ID saved by ESC Flash, and the other is the communication address ID combined with ESC PWM (ESC will not save the node ID value saved by ESC Flash will be restored after power failure).

7.1.1 ESC node parameters

Modify ESC parameters by referring to the host page

The screenshot shows the 'ESC Setting' interface. On the left is a sidebar with options: Home, ESC Setting (selected), Encoder, Data Plot, Firmware, Analysis, and Setting. The main area is titled 'Board Param' and contains a list of parameters with input fields and 'Read'/'Write' buttons. The 'Node ID' parameter is highlighted with a red box. The 'Config Name' is 'foc_default_config'. Other parameters include Uart Baudrate (115200), CAN Baudrate (1000K), ESC Index (0), and various Status Upload Periods (0 ms).

Parameter	Value	Read	Write
Device Name	SE1460D	Read	Write
Config Name	foc_default_config	Read	Write
Node ID	32	Read	Write
Uart Baudrate	115200	Read	Write
CAN Baudrate	1000K	Read	Write
ESC Index	0	Read	Write
Status Upload Period	0 ms	Read	Write
Status1 Upload Period	0 ms	Read	Write
Status2 Upload Period	0 ms	Read	Write

7.1.2 ESC communication addressing

Here, the PWM of the ESC is required for addressing (flight control needs to determine the address ID of the ESC through the PWM channel), and the addressing command will only respond to the addressing command when the PWM pulse width received by the ESC is within the range of 300~500us. The address ID is not saved. If the ESC is powered off, the node ID value saved by the ESC Flash is restored.

Addressing can be done using both the ESC addressing directive (208) and the broadcast ESC addressing directive (20030), but you need to know the ESC ID to use the 208 directive.

7.1.2.1 ESC addressing

	Request	Response
Data ID	208	
Data signature	0x1BDDF796A3CF4723	
Data format	uint16 esc_id	uint8 esc_id uint8 valid_flag
Example	Priority 0, host ID is 1, ESC ID is 0x20, PWM throttle pulse width needs to be within 300~500us CAN ID CAN DATA 00D0A081 0x21 0xC0 00D001A1 0x21 0x01 0xC0	

7.1.2.2 Broadcast electrical addressing

Data ID	20030
Data signature	0x5807C97610B3F91C
Data format	uint8 new_esc_id #esc id The range is from 1~127
Example	The priority is 0, the flight control ID is 0x01, and the new ID is set to 0x21. When the received PWM is between 300 and 500us, the ESC will actively send a response packet to obtain the broadcast addressing status instruction. CAN ID CAN DATA 004E3E01 0x21 0xC0 00CF01A1 0x21 0x01 0xC0 Addressing success , response package of ESC

7.1.2.3 Gets broadcast addressing status

	Request	Response
Data ID	207	
Data signature	0x1BDDF796A3CF4723	
Data format	none	uint8 alloc_id_state #0, success; 1 can id error
Example	Priority 0, host ID is 1, ESC ID is 0x21 CAN ID CAN DATA 00CFA181 0xC0 00CF01A1 0x00 0xC0	

8 ESC Firmware upgrade

8.1 Firmware file composition

0~1 byte	2~3 byte	4~5 byte	6~7 byte	8~31 byte	32~...
Hardware_id	Not In Use	File Size KB	File CRC	Not In Use	Firmware data

The first 32 bytes of the file contain the firmware information, which needs to be parsed before the firmware upgrade.

8.2 Device restart

	Request	Response
Data signature	5	
Data signature	0x569E05394A3017F0	
Data format	uint40 magic_number //0xACCE551B1E	bool ok
Frame ID field	00 05 A0 81	00 05 01 A0
Example	Priority 0, host ID is 1, ESC ID is 0x20 CAN ID CAN DATA 0005A081 0x1E 0x1B 0x55 0xCE 0xAC 0xC0 000501A0 0x80 0xC0 //这里注意是 0x80, 不是 0x01	

8.3 Keep the Bootloader status

	Request	Response
Data signature	200	
Data signature	0x8BA32EBA566F5713	
Data format	无	uint8 mode #1 APP, 2 Bootloader
Example	Priority 0, host ID is 1, ESC ID is 0x20 CAN ID CAN DATA 00C8A081 0xC0 00C801A0 0x02 0xC0	

8.4 Start upgrade

	Request	Response
Data ID	201	
Data signature		
Data format	uint16 hardware_id uint16 filse_size_kb uint16 file_crc	uint8 state #0 Firmware is normal, 1 firmware is incorrect
Frame ID field	00 C9 A0 81	00 C9 01 A0

Example	Priority 0, host ID is 1, ESC ID is 0x20	
	CAN ID	CAN DATA
	00C9A081	Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 0xC0
	00C901A0	0x00 0xC0

8.5 Firmware transfer

Note that the data transferred here starts with the 32nd byte of the firmware file and continues to the end of the file.

index is the index of each transfer, counting from 0.

	Request	Response
Data ID	202	
Data signature	0xF8D50AFD350A8FBB	
Data format	uint16 index uint8 data[256]	uint8 state #0 Successful, 1 Data error uint16_t next_index
Example	Priority 0, host ID is 1, ESC ID is 0x20	
	CAN ID	CAN DATA
	00CAA081	CRC_L CRC_H Byte3 Byte4 Byte5 Byte6 0x80
	00CAA081	Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 0x20
	00CAA081	Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 0x00
	...	
	00CAA081	Byte1 Byte2 Byte3 Byte4 Byte5 Byte6 0x00
	00CAA081	Byte1 0x60
	00CA01A0	0x00 0x02 0x00 0xC0

8.6 End Upgrade

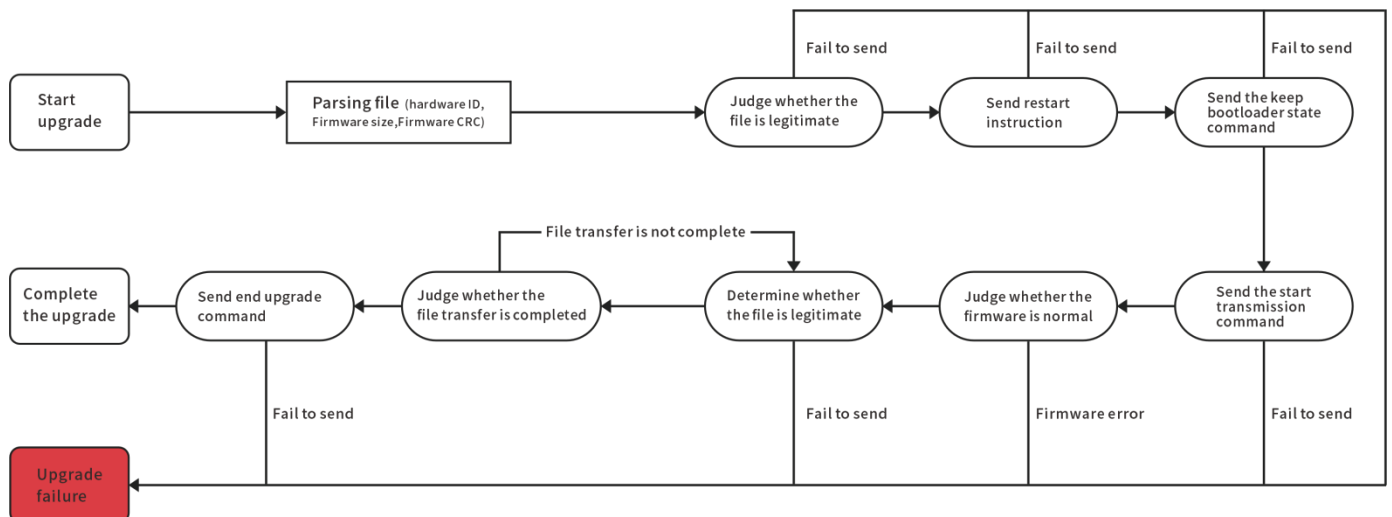
	Request	Response
Data ID	203	
Data signature	0xD1AC3920854F5803	
Data format	none	uint8 state #0 upgrade succeeds. 1 upgrade fails
Frame ID field	00 CB A0 81	00 CB 01 A0

Example	Priority 0, host ID is 1, ESC ID is 0x20	
	CAN ID	CAN DATA
	00CBA081	0xC0
	00CB01A0	0x00 0xC0

8.7 Firmware upgrade flowchart

Note: It is recommended that if no reply packet is received within 150ms, it is considered that the transmission fails and the command needs to be resend. The recommended number of resends is 10.

When sending firmware data, you need to read the firmware data according to the next_index of the returned firmware packet



9 Appendix 1 Definition of Error codes

9.1 Self-check error code

Bit	Fault type	Reason analysis	Level
0	Motor line current output abnormal 01	Hardware failure, chip problem or short circuit welding	High
1	Total current output abnormal	Hardware failure, resistance problem or short circuit welding	High
2	Motor line voltage abnormal	Hardware failure, resistance problem or short circuit welding	High
3	Electrolytic capacitor NTC output abnormal	Hardware failure, resistance problem or short circuit welding	High
4	MOS Tube NTC output abnormal	Hardware failure, resistance problem or short circuit welding	High
5	Bus voltage over voltage or low voltage	The power supply voltage is too high or too low	High
6	The bus voltage sampling abnormal	Hardware failure, resistance problem or short circuit welding	High
7	motor wire Loop impedance is too low	The motor line circuit is short circuit or the motor coil is short circuit	High
8	motor wire Loop impedance is too large	The motor wire contact is not good or the motor coil is open	High
9	Motor line voltage detection circuit abnormal 1	Hardware failure, resistance problem or short circuit welding	High
10	Motor line voltage detection circuit abnormal 2	Hardware failure, resistance problem or short circuit welding	High
11	Motor line current detection circuit abnormal 02	Hardware failure, resistance problem or short circuit welding	High
12	undefined		
13	Software and hardware versions are incompatible	Wrong firmware burning	High
14	Boot loader unsupported	Wrong firmware burning	High
15	Undefined		

9.2 Run code error

Bit	Fault type	Reason analysis	Level
0	Occurs Over current/short circuit protection occurs	Motor line is short-circuited or blocked	High
1	Occurs Locked-rotor protection occurs	The motor is blocked	High
2	over-temperature protection	Motor overload operation or poor heat dissipation	High
3	PWM throttle lost pulse	PWM valve line is in poor contact, or the valve line is disturbed	High
4	No load	Motor is not under load	low
5	Throttle saturation	Throttle saturation appears, indicating that the voltage is low or the motor KV is not enough	middle
6	Occurs Over voltage protection	A power supply test was used, you can try a battery powered test	High
7	Occurs Voltage drop	ESC detects a sudden surge in voltage during operation, which may be a loose power plug	High
8	Communication throttle loss	The communication throttle packet transmission is unstable, or the interval is greater than 60ms	low
9~15	Undefined		